Qminers

matfyz

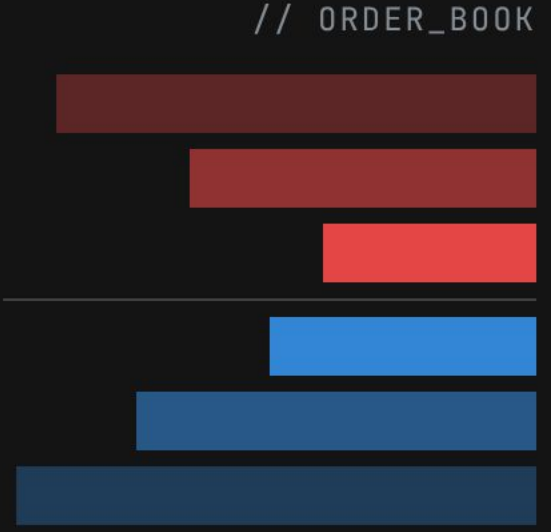
SYS.STATUS // GETTING STARTED

ALGO TRADE ARENA

> From an empty repo.
> To your first strategy.

NOVEMBER 07, 2026

IMPAKT, MFF UK PRAGUE



// TODAY

Six steps

01
Get the repo
Private copy, cloned

02
Install uv
It brings Python

03
First run
Sync, unzip, backtest

04
The exchange
Basics, book, matching, latency, score

05
Backtest explorer
See what your strategy did

06
First strategy
A trend follower, line by line

// STEPS 01-03

GET
SET UP

private repo · uv · first backtest

// 01 · GET THE REPO

Make your own private copy

1

Open

`github.com/QminersOrg/qminers-algo-arena`

→

2

Click

Use this template

→ Create a new repository

→

3

Owner: you

Name: anything

◉ **Private**

→

4

Click

Create repository

Why not “Fork”?

A fork of a public repo stays public: your strategy would be visible to everyone.

In a team?

One person creates it, then Settings → Collaborators → add teammates.

// 01 · GET THE REPO

Clone it to your computer

```
git clone https://github.com/<you>/<your-repo>.git
cd <your-repo>
```

No Git yet? Install it from git-scm.com

Asked to sign in? It's a private repo: use the browser pop-up, an SSH key, or GitHub Desktop → Clone

// WHAT YOU GET

strategies/
your code + two examples

data/
market data (CSV)

logs/
backtest results

simulator/
the exchange (same code as scoring)

explorer/
the backtest explorer app

// 02 · INSTALL UV

One tool. It brings Python.

// MACOS / LINUX

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

// WINDOWS (POWERSHELL)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Check

New terminal → **uv --version**

No Python install needed

uv downloads Python 3.13 for this project by itself

// 03 · FIRST RUN

Install packages, unzip the data

```
uv sync                                          # Python 3.13 + packages → .venv/
uv run python -m gzip -d data/sample.csv.gz      # the simulator reads plain CSV
```

sample.csv

1 synthetic hour, trending prices. A playground to check your code runs.

plain CSV only

The simulator can't read .gz: unzip every data file first.

any OS

Same two commands on Windows, macOS and Linux.

// 03 · FIRST RUN

Run your first backtest

```
uv run exchange-simulator \
  --strategy strategies/trend_follower.py \
  --backtest data/sample.csv \
  --log-dir  logs/sample \
  --progress
```

Only need the score? Add --no-logs

Writes just result.json and pnl.csv: much faster

```
fills=305
breach_penalty=0.00
pnl=1878.00
```

result.json

pnl = your score on this data

pnl.csv

PnL every second

events · trades · book_deltas

full logs for the explorer

// BEFORE STEP 04

TRADING BASICS

one instrument · order book · orders · PnL

// BASICS

What we simulate

One instrument. A tape of other traders' orders is replayed through an exchange, your bot trades alongside it, and your profit is the score.

1
instrument, anonymised: no name, no news, only its orders

1 h
of synthetic orders in sample.csv to start on

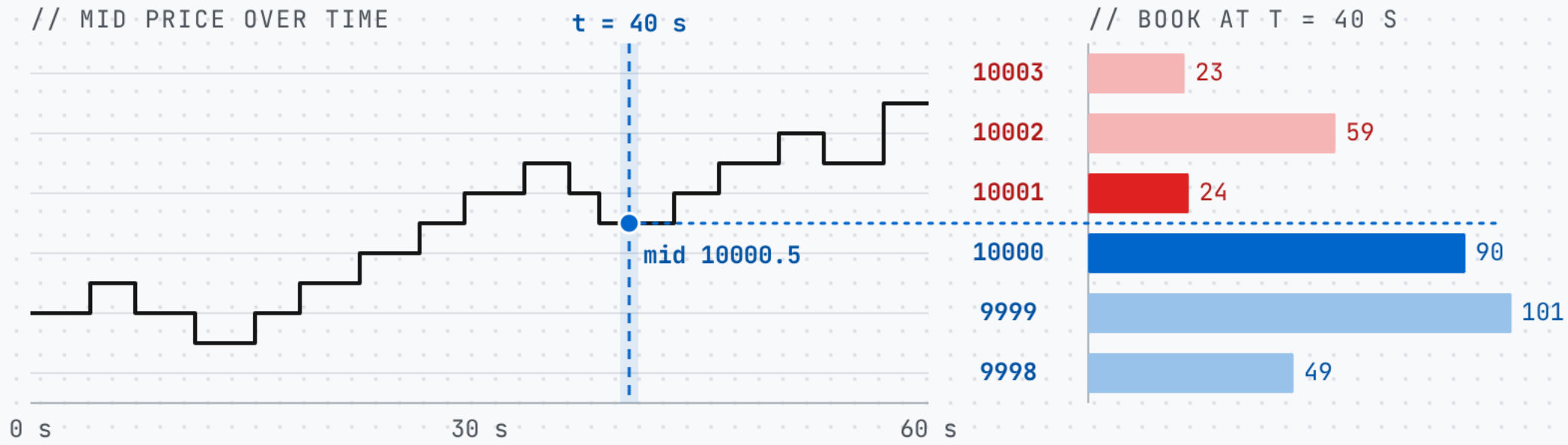
5 ms
from your call to the exchange acting on it

±100
lots you can hold without a penalty

> **Your job:** buy lower than you sell, and never hold too much.

// BASICS

Bid, ask, spread, mid



BID
a price someone waits to buy at

ASK
a price someone waits to sell at

SPREAD
best ask − best bid: 1 tick here

MID
halfway between them: the price line

// BASICS

Two order types

// LIMIT · TAPE CODE G

Wait for your price

```
BUY 10 @ 9999
```

Nobody sells at 9999 or less, so it rests in the book as a new bid and waits.

Maker: you earn the spread

// IOC · TAPE CODE F

Trade now or never

```
BUY 10 @ 10001
```

Takes up to 10 lots from asks at 10001 or less right now. Whatever is left is cancelled.

Taker: you pay the spread

No separate “market order”: an IOC priced through the book does that job.

// BASICS

What happens to orders

// EVENT	// MEANING	// ON THE TAPE	// YOUR CALL
Insert	A new order arrives: it trades if it can, else it rests	`Insert`	`send_order()`
Cancel	A resting order is pulled from the book	`Cancel`	`cancel_order()`
Modify	A resting order is made smaller and keeps its place	—	`modify_order()`
Fill	Two orders trade. Only part of an order: a partial fill	—	`none: you're told`

The tape is only inserts and cancels. Every trade is the exchange matching them.

// BASICS

Position and PnL

BUY 10 @ 10001

inventory +10 (long)

cash -100,010

→

SELL 10 @ 10003

inventory 0 (flat)

cash +20

→

PnL +20

cash + value of what you still hold - penalty

LOT

one unit of size; sizes are whole lots

INVENTORY

lots bought − lots sold, your position

LONG · SHORT · FLAT

inventory above, below, at zero

CASH

money in from sells − money out on buys

// BASICS

Cheat sheet

the market	meaning
Tick	smallest price step
Lot	smallest size unit
Bid / ask	buy / sell price waiting in the book
Touch	best bid and best ask
Spread	best ask − best bid
Level	all orders at one price
Depth	levels you see: 20 per side

you	meaning
Maker / taker	rests in the book / trades into it
Fill	a trade of your order, full or partial
Inventory	lots bought − lots sold
Long / short / flat	inventory > 0 / < 0 / = 0
Cash	money from sells − money for buys
Markout	inventory valued against the book
PnL	cash + markout − penalty: the score

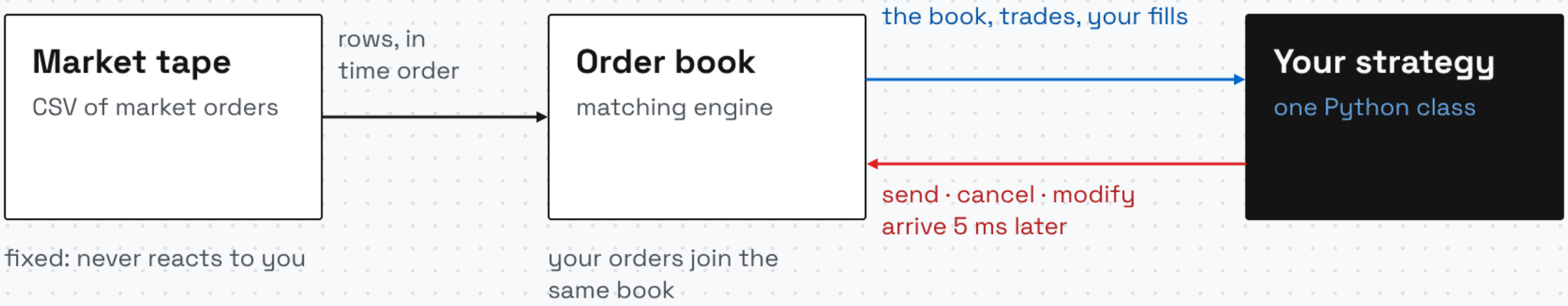
// STEP 04

HOW THE EXCHANGE WORKS

tape · order book · matching
latency · limits · score

// 04 · THE EXCHANGE

The big picture



// 04 · THE EXCHANGE

The tape: one row, one action

Time	Operation	OrderId	Side	Volume	Price	Lifespan
0.003823	Insert	1000001	B	43	9999	G
0.491183	Cancel	1000001				
seconds	Insert or Cancel	which order	B buy · A sell	lots	ticks	G rests · F IOC

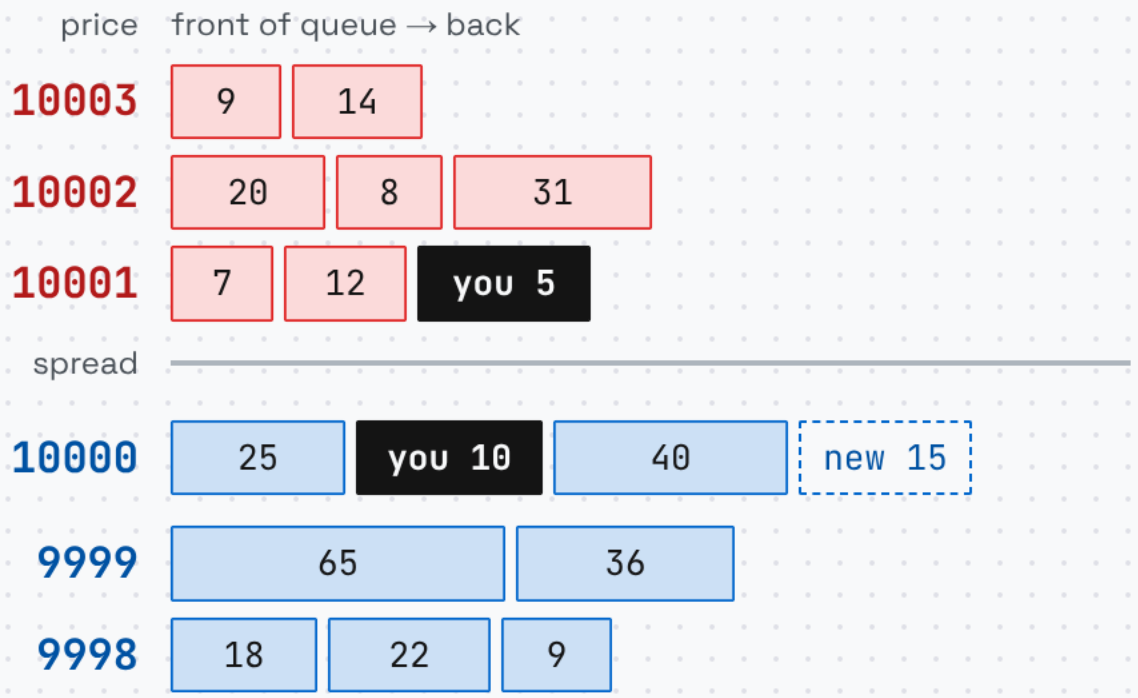
53,758
rows in sample.csv

1 hour
of synthetic market activity

Fixed
the tape never reacts to your orders

// 04 · THE EXCHANGE

The order book: a queue at every price



← best ask (lowest sell)

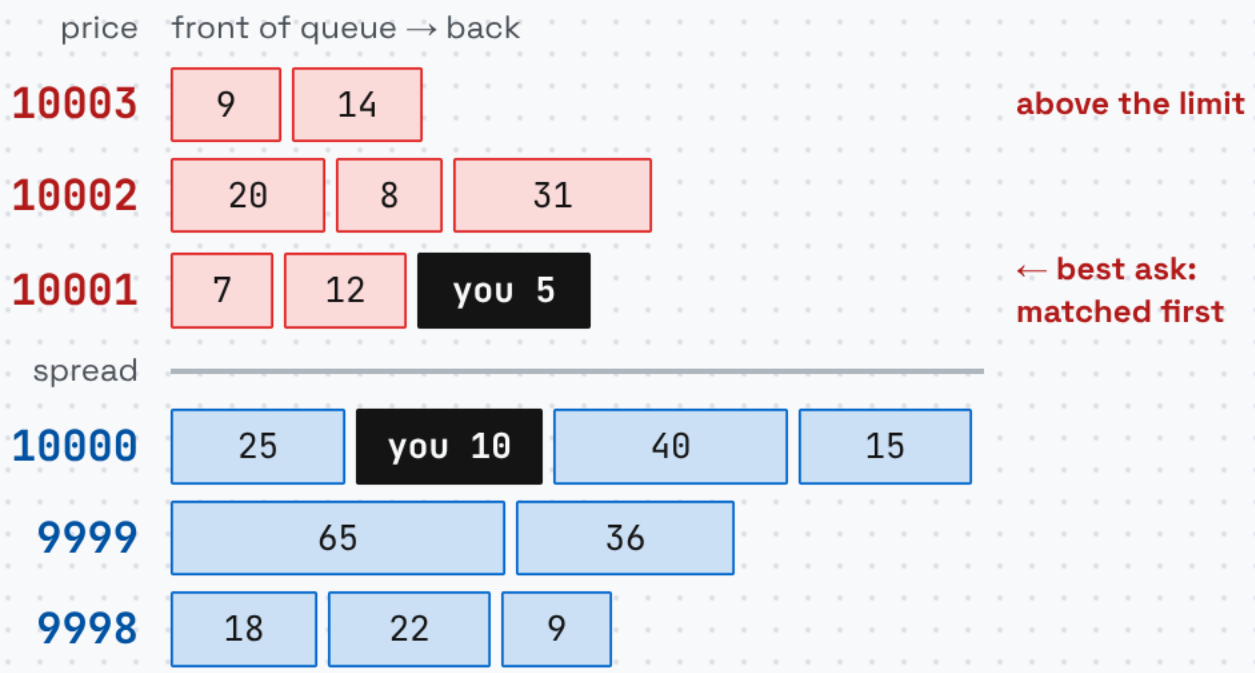
10001 − 10000 = 1 tick

← best bid (highest buy)

a new order joins the back

// 04 · MATCHING · THE ORDER ARRIVES

A big buyer arrives



above the limit

← best ask: matched first

// INCOMING IOC

BUY 100 @ 10002

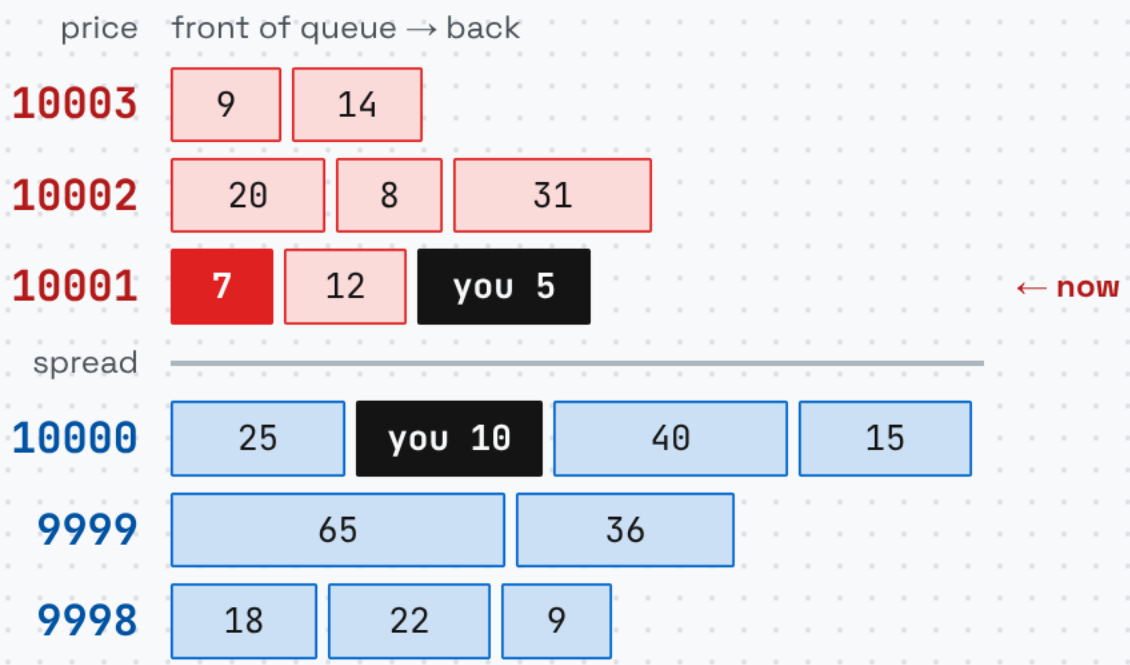
LEFT TO FILL **100**

// TRADES

none yet

// 04 · MATCHING · TRADE 1 OF 6

Trade 1: 7 lots at 10001



// INCOMING IOC

BUY 100 @ 10002

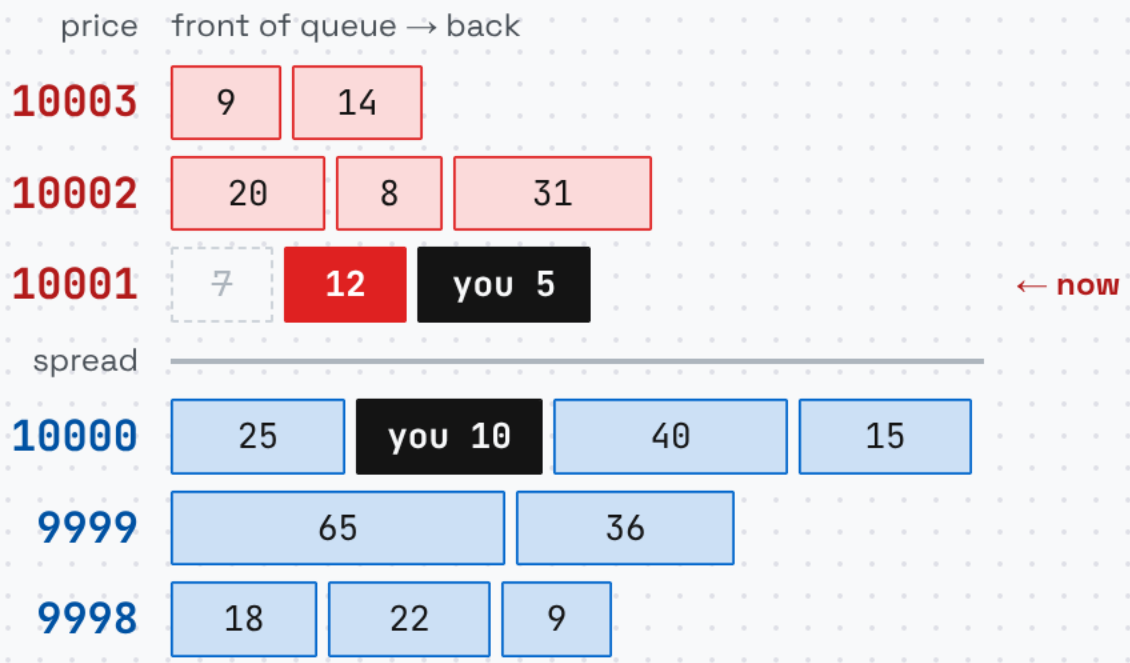
LEFT TO FILL **93**

// TRADES

#1 7 @ 10001 market

// 04 · MATCHING · TRADE 2 OF 6

Trade 2: 12 lots at 10001



// INCOMING IOC

BUY 100 @ 10002

LEFT TO FILL **81**

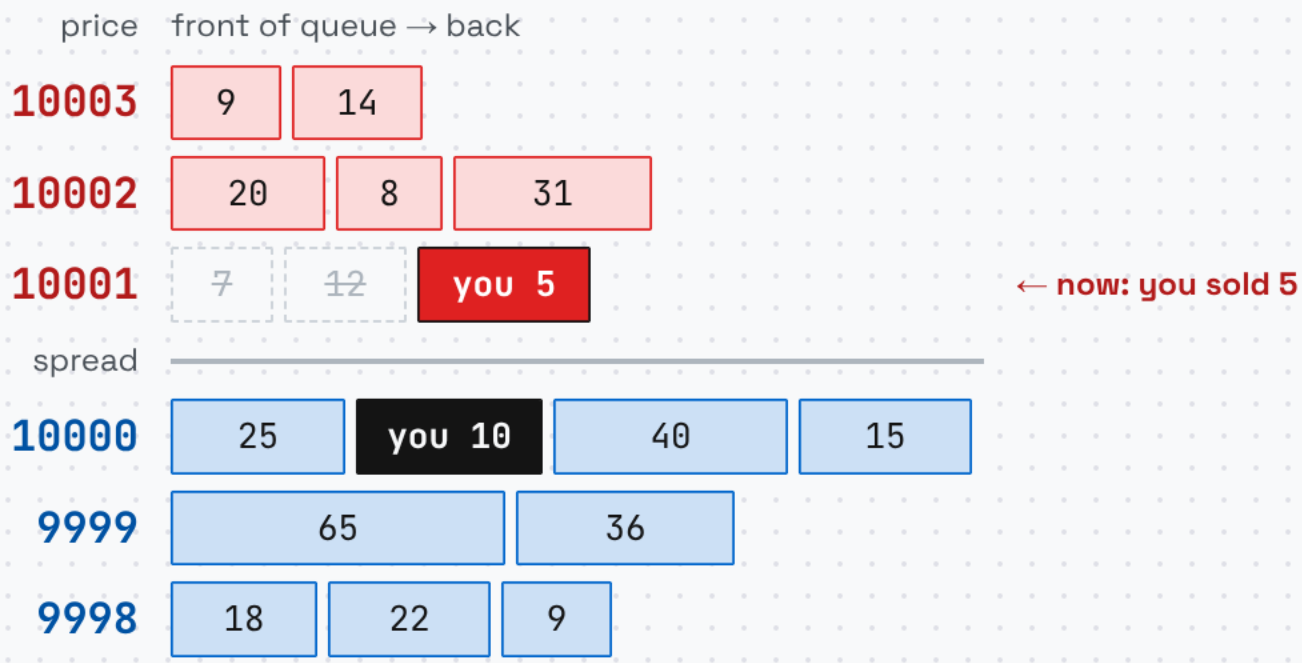
// TRADES

#1 7 @ 10001 market

#2 12 @ 10001 market

// 04 · MATCHING · TRADE 3 OF 6

Trade 3: your 5 lots at 10001



← now: you sold 5

// INCOMING IOC

BUY 100 @ 10002

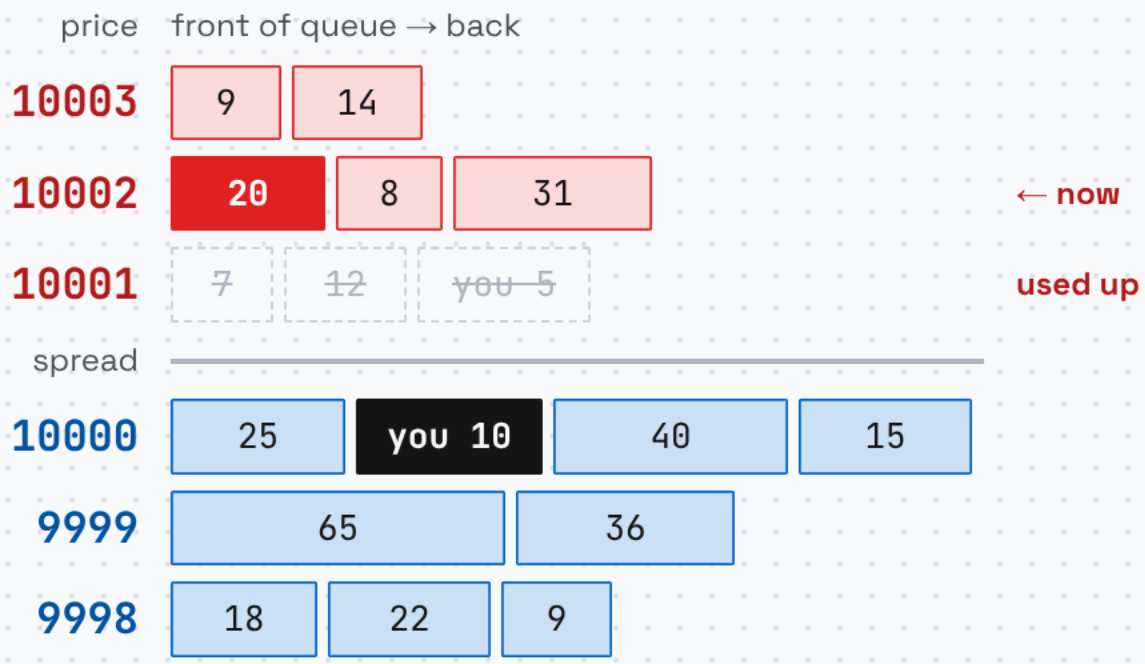
LEFT TO FILL **76**

// TRADES

```
#1    7 @ 10001   market
#2   12 @ 10001   market
#3    5 @ 10001   you
```

// 04 · MATCHING · TRADE 4 OF 6

Trade 4: 20 lots at 10002



// INCOMING IOC

BUY 100 @ 10002

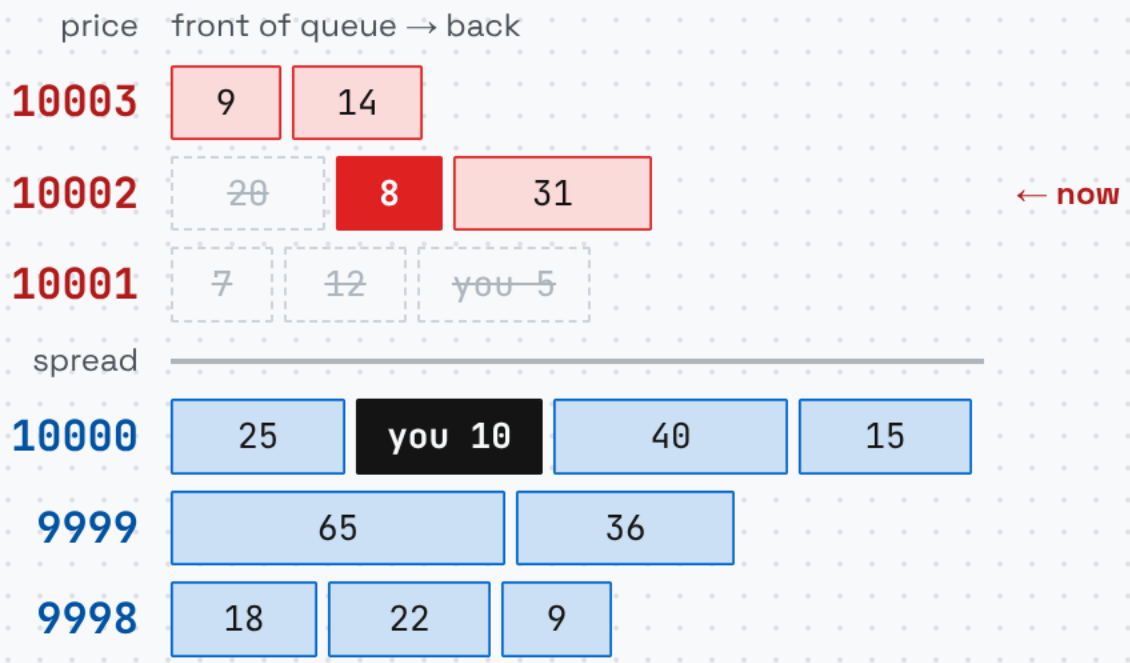
LEFT TO FILL **56**

// TRADES

#1	7	@ 10001	market
#2	12	@ 10001	market
#3	5	@ 10001	you
#4	**20**	**@ 10002**	**market**

// 04 · MATCHING · TRADE 5 OF 6

Trade 5: 8 lots at 10002



// INCOMING IOC

BUY 100 @ 10002

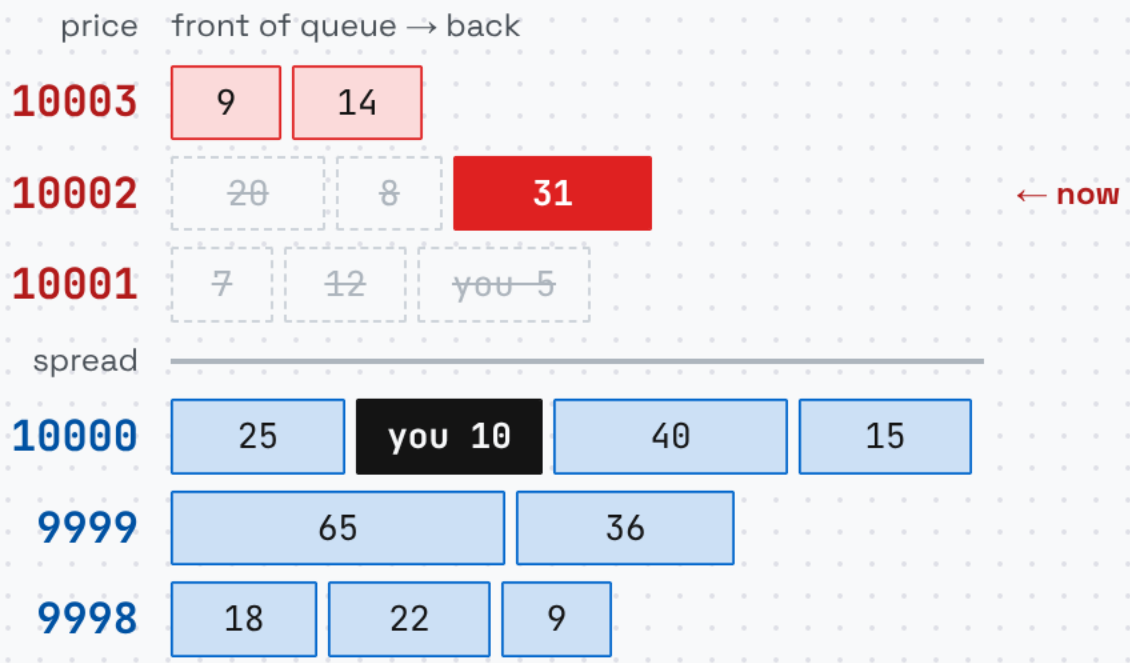
LEFT TO FILL **48**

// TRADES

#1	7 @ 10001	market
#2	12 @ 10001	market
#3	5 @ 10001	you
#4	20 @ 10002	market
#5	**8 @ 10002**	**market**

// 04 · MATCHING · TRADE 6 OF 6

Trade 6: 31 lots at 10002



// INCOMING IOC

BUY 100 @ 10002

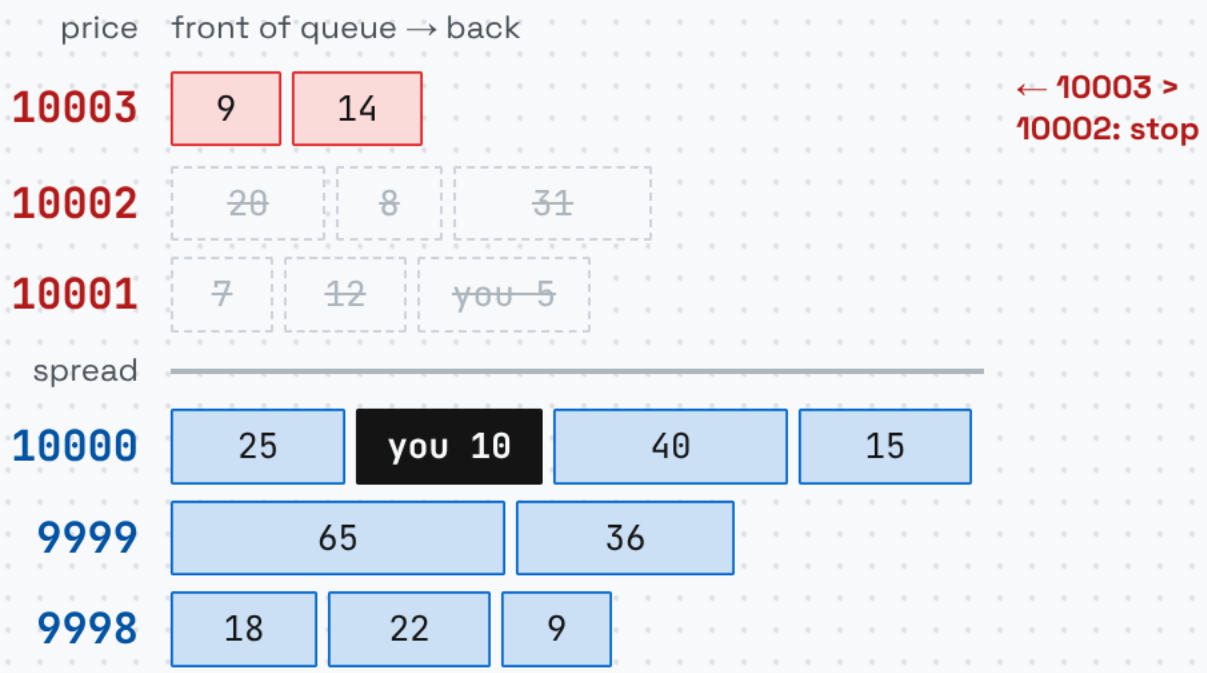
LEFT TO FILL **17**

// TRADES

#1	7 @ 10001	market
#2	12 @ 10001	market
#3	5 @ 10001	you
#4	20 @ 10002	market
#5	8 @ 10002	market
#6	**31 @ 10002**	**market**

// 04 · MATCHING · THE END

17 left, nothing within the limit



// INCOMING IOC

BUY 100 @ 10002

LEFT TO FILL **17**

if IOC

the 17 are cancelled. Done.

if LIMIT

the 17 rest at 10002: the new best bid

// 04 · MATCHING

Four more rules

Market orders fill yours

Your resting orders wait in the same queues

Yours never trade with yours

Your buy skips past your own sell

No fees

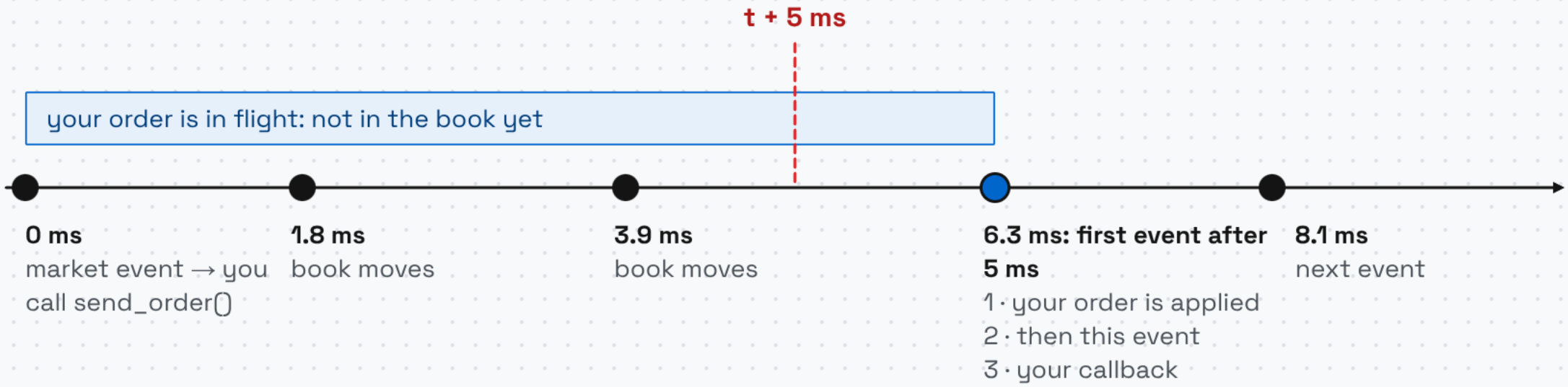
You pay only the spread you cross

Whole numbers only

Prices in ticks, sizes in lots

// 04 · LATENCY

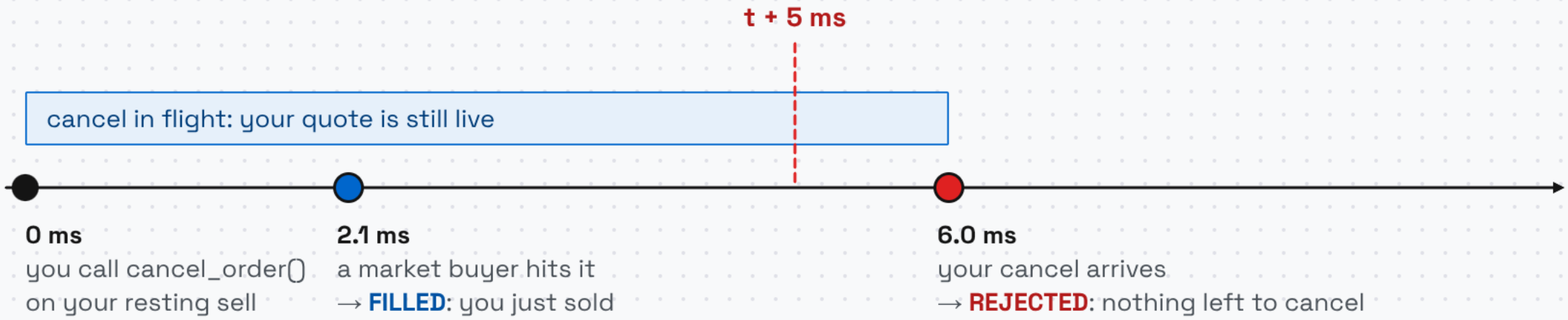
Every order arrives 5 ms late



In a busy market, **several events** can slip past every order you send.

// 04 · LATENCY

Cancels are late too



Sent a cancel ≠ cancelled. Count inventory from **fills**, not from what you sent.

// 04 · THE LOOP

For every row of the tape, in this order

1. **Your orders whose 5 ms are up**
   may trade at once · fires on_private_order_update

2. **The market row**
   may fill your orders · fires on_private_order_update

3. **on_order_book_change**
   new book + all trades since your last call

53,758

times on sample.csv alone. Keep your callbacks light.

// 04 · WHAT YOU SEE

Two callbacks

`on_order_book_change(snapshot, trades)`

after every market event

`snapshot.bids · snapshot.asks`

every order, best price first, 20 ticks deep

`.price · .quantity · .owner`

owner is 'market' or 'user' (you)

`trades`

all trades since the last call, each with price, quantity and book_side

`on_private_order_update(event)`

instantly, only for your orders

`event.status`

what just happened to the order

`.filled_quantity · .fill_price`

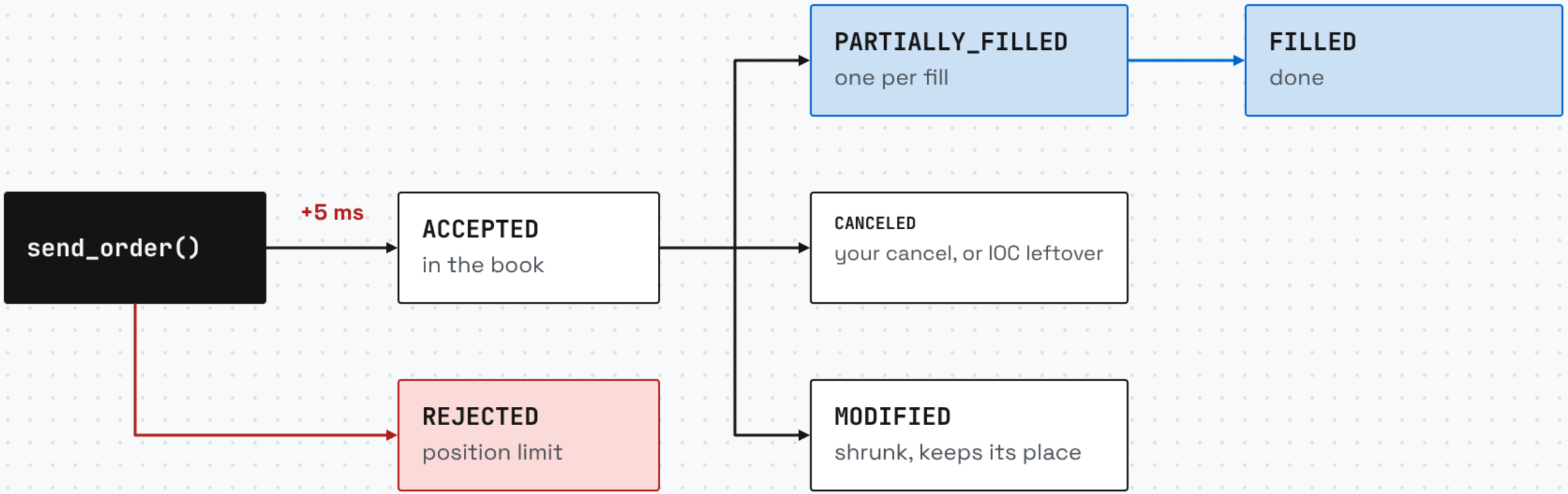
one event per fill

`.remaining_quantity · .order_id`

what's left, and which order

// 04 · ORDER LIFE

What can happen to your order



// 04 · WHAT YOU CAN DO

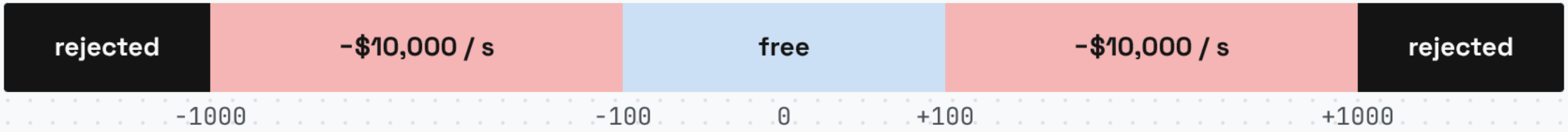
Four calls

`self.send_order(OrderRequest(side, price, quantity, order_type))`	returns the order id at once
`self.cancel_order(order_id)`	pull your resting order
`self.modify_order(order_id, new_quantity)`	shrink only, keeps queue place
`self.now_us()`	simulation time in µs

`Side.BUY · Side.SELL` `OrderType.LIMIT · OrderType.IOC`

// 04 · LIMITS

Your inventory has two walls



When an order arrives

`|I + s·(R + q)| > 1000 → REJECTED`

I inventory · R your resting size on that side · q order size · s = +1 buy, −1 sell

While you hold it

`|I| > 100 → -$10,000 per second`

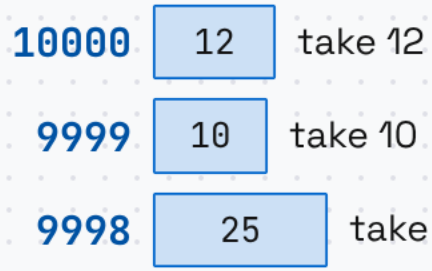
charged for the exact time spent outside ±100, taken off your PnL

// 04 · SCORE

Your score is PnL

PnL = cash + markout(I) - penalty

Holding **I = +30**? Markout sells it into the resting market bids:



12·10000 + 10·9999 + 8·9998 = **299,974**

// EXAMPLE

Bought 30 at 10001

cash -300,030
markout +299,974

PnL -56

// 04 · SUBMISSIONS

What a submission may use

Allowed

Python 3.13 standard library

numpy · pandas · scipy · scikit-learn

```
from simulator.models import …
```

```
from simulator.strategy import BaseStrategy
```

Rejected before it runs

```
os sys io socket subprocess threading importlib
inspect gc …
```

```
open eval exec getattr type
```

dunder attributes (__init__ is fine) · str.format

any other simulator import

One .py file, one BaseStrategy subclass, no constructor arguments.

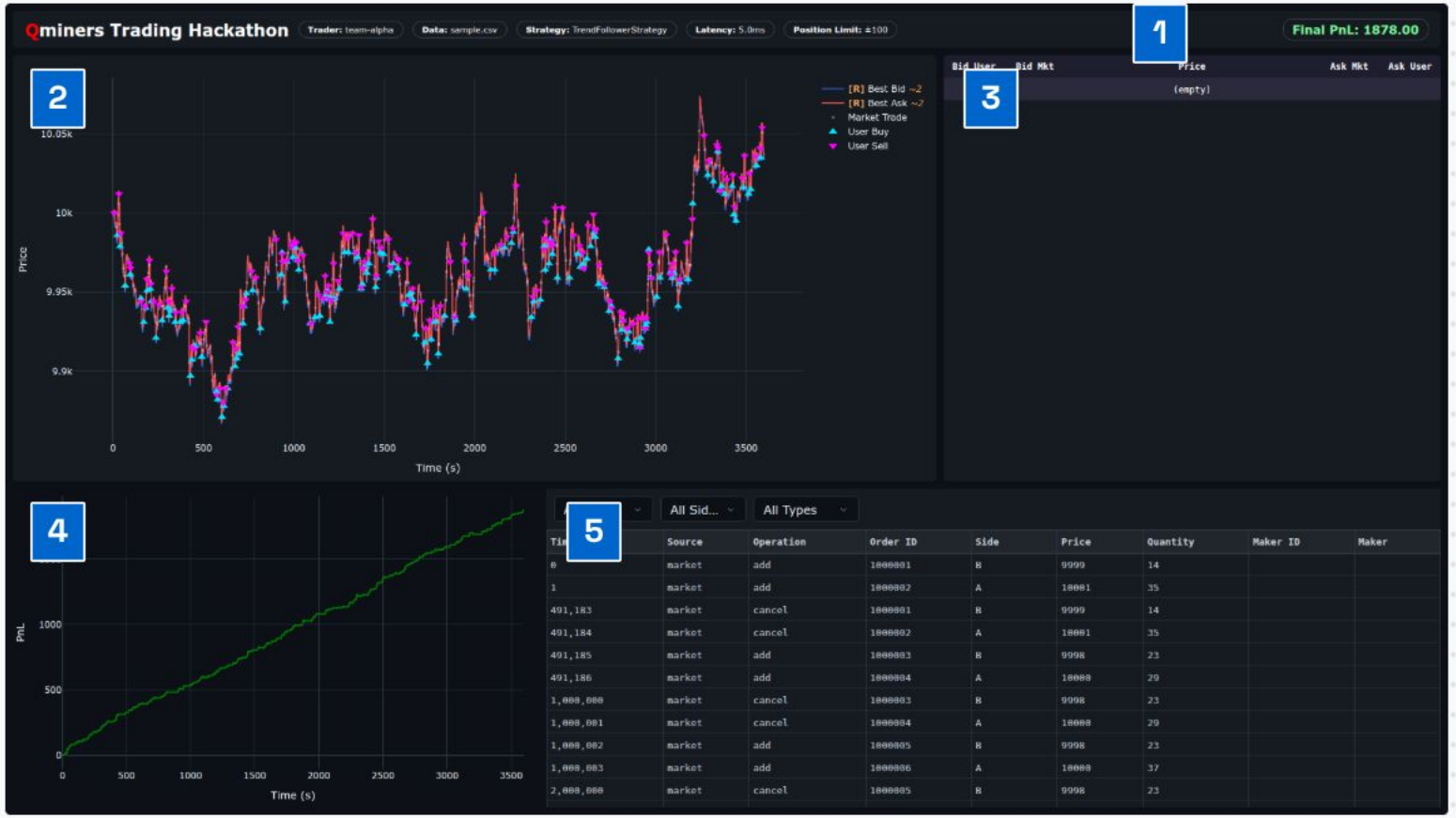
// STEP 05

BACKTEST EXPLORER

> every order, fill and PnL tick of one run

// 05 · EXPLORER

One screen per run

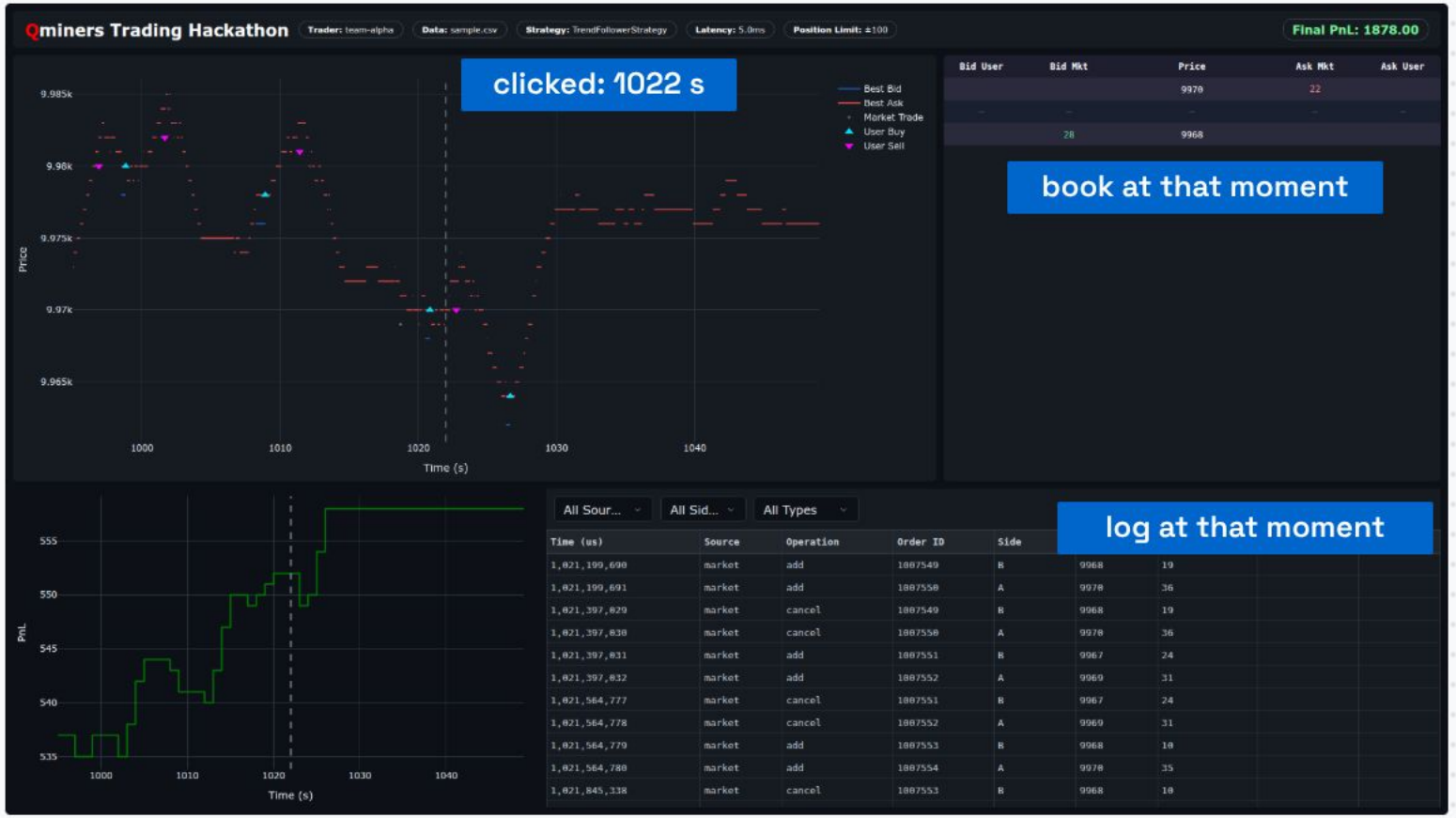



```
uv run backtest-explorer
  --log-dir logs/sample
→ 127.0.0.1:8050
```

1. Final PnL: your score on this data
2. Bid/ask, trades, your ▲ buys ▼ sells
3. The book at the selected moment
4. PnL every second, penalty included
5. Every event, yours and the market's

// 05 · EXPLORER

Zoom in, click, travel in time



Zoom one chart

the other follows

Click a point

book and log jump to that moment

Click a log row

crosshair on both charts

Filter the log

Source: User shows only yours

// 05 · EXPLORER

Four questions to ask it

Filled where I expected?

Compare ▲ ▼ with the bid and ask lines

What happens after a fill?

Price running against you means you were picked off

Where does PnL drop?

Click the cliff, read the log at that moment

Are orders failing?

Type filter: Add Failed, Cancel Failed

Needs full logs: it can't read a `--no-logs` run.

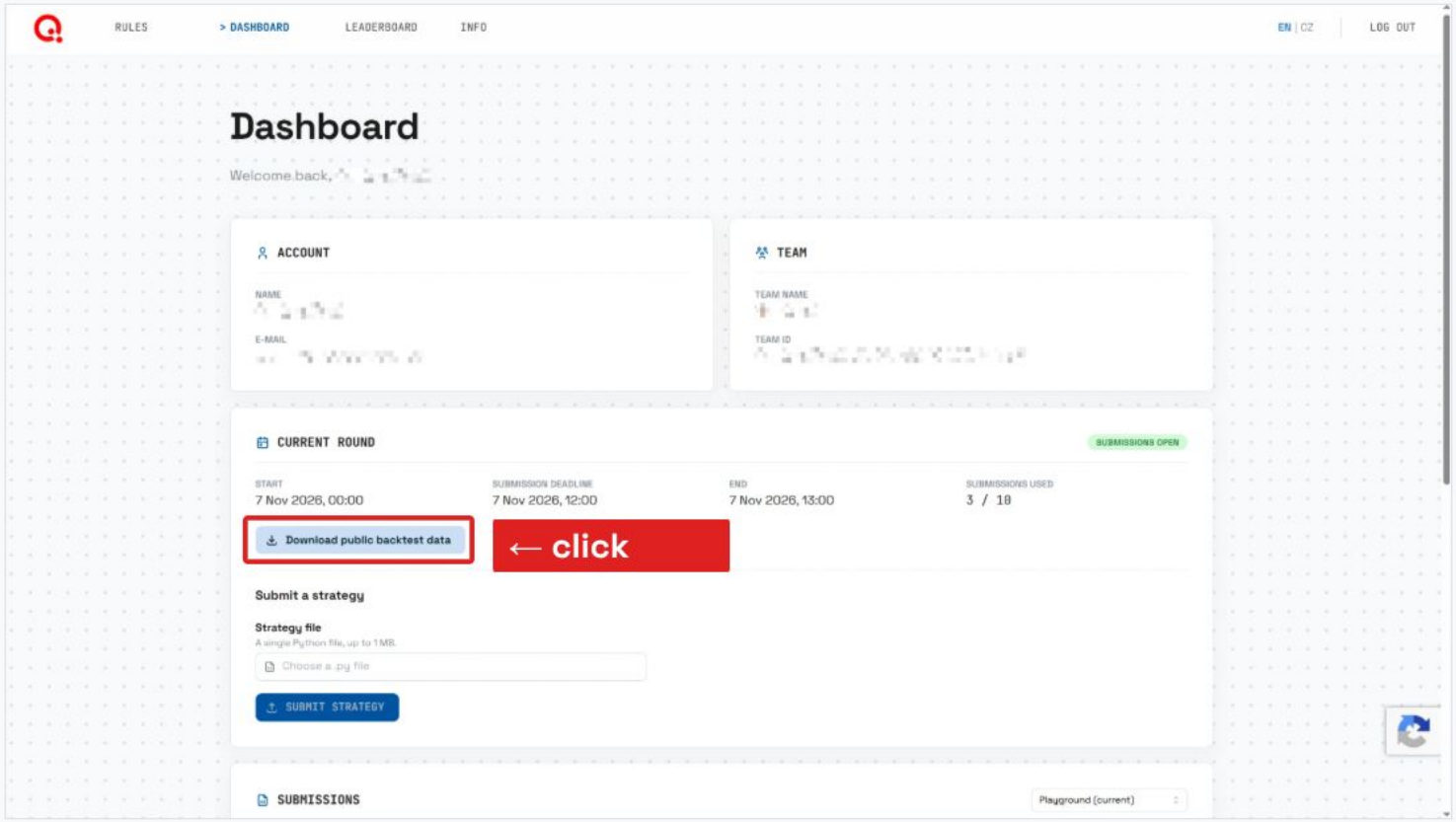
// STEP 06

YOUR FIRST STRATEGY

> strategies/trend_follower.py, from an empty file

// 06 · YOUR FIRST STRATEGY

sample.csv is also on the website



1. Log in at arena.qminers.com
2. Dashboard → **Download public backtest data**
3. Save it in data/ and unzip it as in step 03

> Everything in this section runs on that file.

// 06 · THE IDEA

Trade the way the market just traded

// TRADES SINCE LAST CALL

+12 · a buyer took the ask

+5 · a buyer took the ask

-4 · a seller hit the bid

→

net flow

+13

→

// SEND

BUY 1 @ ask

IOC, at the best ask

One order in flight

Wait for its answer before sending the next

Never the same side twice

Buy, sell, buy… so inventory stays in {−1, 0, +1}

// 06 · STEP 1 OF 5

Imports, class, state

```
from simulator.models import (
    OrderBookSnapshot, OrderRequest, OrderStatus,
    OrderType, PrivateOrderEvent, Side, Trade,
)
from simulator.strategy import BaseStrategy

class TrendFollowerStrategy(BaseStrategy):
    def __init__(self) -> None:
        super().__init__()
        self._order_in_flight: bool = False
        self._current_order_side: None | Side = None
```

One file, one BaseStrategy subclass, no arguments

Two pieces of state: busy? and last side traded

// 06 · STEP 2 OF 5

Measure the flow

```
    def on_order_book_change(self, snapshot: OrderBookSnapshot, trades: list[Trade]) → None:
        if self._order_in_flight:
            return

        # Calculate how much was traded in the market
        side_sign = {'ask': 1, 'bid': -1}
        total_traded = sum(t.quantity * side_sign[t.book_side] for t in trades)
```

Busy? Skip. The answer takes ≥ 5 ms.

'ask': a buyer took the ask → +

'bid': a seller hit the bid → −

Pick a side and a price

```
        # Pick order side and price based on direction of trading and book
        if total_traded == 0:
            return
        elif total_traded > 0:
            if not snapshot.asks or self._current_order_side == Side.BUY:
                return
            self._current_order_side, best_price = Side.BUY, snapshot.asks[0].price
        else:
            if not snapshot.bids or self._current_order_side == Side.SELL:
                return
            self._current_order_side, best_price = Side.SELL, snapshot.bids[0].price
```

No net flow, or empty side: do nothing

Same side as last time: skip, inventory stays in {−1, 0, +1}

// 06 · STEP 4 OF 5

Send it

```
        # Send order
        self.send_order(
            OrderRequest(
                side=self._current_order_side,
                price=best_price,
                quantity=1,
                order_type=OrderType.IOC,
            )
        )
        self._order_in_flight = True
```

IOC at the touch: trade now or not at all

Book moved in 5 ms? It misses: CANCELED

Mark busy until we hear back

// 06 · STEP 5 OF 5

Listen for the answer

```
    def on_private_order_update(self, event: PrivateOrderEvent) → None:
        if event.status == OrderStatus.FILLED:
            self._order_in_flight = False
        elif event.status == OrderStatus.REJECTED or event.status == OrderStatus.CANCELED:
            self._order_in_flight = False  # free the slot on a miss too
            self._current_order_side = Side.BUY if self._current_order_side == Side.SELL else Side.SELL
```

FILLED

We traded. Free the slot; the side stays, so next time we go the other way.

CANCELED or REJECTED

We didn't trade. Free the slot and undo the side, so the same side may retry.

Without the green line the first missed IOC leaves the strategy waiting forever.

// LIVE DEMO

SUBMITTING YOUR STRATEGY

arena.qminers.com → Dashboard → Submit strategy

YOUR TURN

> write, backtest, look, submit

STEP_01

Copy the template _

strategies/template_strategy.py → your own file

STEP_02

Backtest it _

--no-logs when you only need the score

STEP_03

Look when surprised _

full logs, then the explorer

STEP_04

Submit _

arena.qminers.com

[Questions?]